Growing prefixes in a fixed remainder program

Research note accompanying the uniform-process construction

October 16, 2026

Abstract

The family of programs in the companion paper's section "Fixed programs with arbitrarily small coefficients" does not yet give one sublogarithmic remainder program. We investigate recomputation, destructive traversal, and recursive reuse of a growing prefix. A concrete finite program of maximum arity three can select arbitrarily deep prefix operands, but the tested computations pay for repeated regeneration. We formalize a limitation of the corresponding cost accounting, and describe a structural direction based on the interfaces between recursive calls. No sublogarithmic algorithm or algorithmic lower bound is established here.

1 The target and the operational model

The desired statement is the existence of one finite nondeterministic recursive program over $(\mathbb{N}, \mathrm{rem}, \mathrm{eq}_0)$ computing the gcd with

$$\max_{1 \le b \le a} c_P(a, b) = o(\log a),$$

where c_P is the minimum remainder count of a convergent computation. Every convergent computation must be correct, and at least one must exist on every input. These are the conventions of [1, §2E]. The theorem of the companion paper's section "Fixed programs with arbitrarily small coefficients" [2] (expanded in the note *Fixed-arity nondeterministic remainder programs* [3]) chooses, for each integer $p \ge 1$, a different fixed program, with generator arity $4^p + 5$ and coefficient at most $3/p$.

The Lean module `ShortGCD/RegisterSmallStep.lean` formalizes the specialized two-stack recursive machine. It proves exact-cost equivalence with finite evaluation trees, including right-to-left argument evaluation, separate zero tests and branches, finite nondeterministic body choice, and a positive divisor requirement. Transition locality is also checked. The machine is for the four-symbol signature (pick, gen, accept, loop) of the fixed-width programs [3]; the program of Section 2 has a different signature and is not covered. A saved caller context survives a child computation, but the child cannot inspect arbitrary saved values. The book itself distinguishes the richer structure used to implement the stacks from the original structure [1, p. 78]. Machine stacks therefore do not supply list primitives to the source program.

2 A single program with unbounded prefix selection

Write $a = r_0, b = r_1, r_2, \ldots$ for the Euclidean sequence. The fixed equations

$$\begin{aligned} \mathrm{prefix}(a, b) &= a, \\ \mathrm{prefix}(a, b) &= b, \\ \mathrm{prefix}(a, b) &= \mathrm{prefix}(b, \mathrm{rem}(a, b)) \end{aligned}$$

can select r_i with exactly $\max(0, i-1)$ remainder calls, whenever the necessary divisors are positive. Thus arbitrary selection depth itself is available in one finite program. Here prefix is the name of a function symbol, kept as in `rebuilding_program.py`: a call walks along the Euclidean sequence and returns one of its terms. Elsewhere, "prefix" means an initial segment $r_0, \ldots, r_h$ of that sequence.

Define the four alternatives

$$\mathrm{near}(u, v) = u \mid v \mid \mathrm{rem}(u, v) \mid \mathrm{rem}(v, u)$$

and the single equation

$$\mathrm{shrink}(a, b, m) = \mathrm{near}\big(\mathrm{rem}(\mathrm{prefix}(a, b), m), \mathrm{rem}(\mathrm{prefix}(a, b), m)\big).$$

Finally let

$$\mathrm{generate}(a, b, m) = m,$$
$$\mathrm{generate}(a, b, m) = \mathrm{generate}(a, b, \mathrm{shrink}(a, b, m)).$$

The symbol generate, also named as in the Python file, iterates shrink and returns a candidate. Use the head and verification wrapper of the fixed-width programs [3]: if $b = 0$, the head returns a; otherwise it generates a candidate starting with $m = b$, and accept returns it only if it is positive and divides both original inputs. Rejection enters a self-loop. All tests are implemented using eq_0 and remainder.

There are seven internal functions including the entry function and rejection loop, all of arity at most three. There are no numeric constants, list operations, or runtime indices in the program. Its syntax is independent of every input and of the planner's prefix length.

Proposition 1 (Correctness and witness cost; proof in this note)**.** *Every convergent computation of this program returns* $\gcd(a, b)$*, and a successful computation exists on every input. For* $a \geq b > 0$*, if* $\gcd(a, b) = r_{E+1}$ *(so Euclid produces it with* E *calls), some successful computation costs at most* $2E + 4$ *calls, including verification.*

Proof. Every common divisor of a, b divides every generated candidate, by induction on evaluations. The wrapper therefore makes every convergent computation correct. A successful computation always exists: if $b = \gcd(a, b)$, stop immediately; otherwise choose the gcd as both prefix operands. Their residues modulo b equal the gcd, so a projection alternative of near returns it. This witness costs at most $2E + 4$ calls, including verification. □

Proposition 1 proves only an $O(\log a)$ upper bound.

Executable check. `rebuilding_program.py` compiles these equations to the finite AST of `register_program.py` [3]. The independently defined right-to-left interpreter executes the same compiled object for every test and planning depth. The planner chooses close residues among the first $h+1$ prefix values; it uses unrestricted arithmetic offline and provides only the nondeterministic choices to the interpreter. Selecting prefix positions i, j costs

$$\max(0, i-1) + \max(0, j-1) + 2 + e, \qquad e \in \{0, 1\},$$

where the two calls produce residues and e records a remainder in near. Both argument occurrences are evaluated, even when the selected values agree. In the table below, "Regeneration calls" sums $\max(0, i-1) + \max(0, j-1)$ over all contractions, and "Local calls" sums the remaining $2 + e$.

The suite checks 270 runs and 270 additional scaled replays with identical choices. It includes all pairs below nine, unordered and zero inputs, larger random and scaled pairs, and a Fibonacci pair. On one seeded 2,048-bit input:

h (planner uses $r_0, \ldots, r_h$)	Contractions	Regeneration calls	Local calls	Total calls
2	538	373	1,566	1,941
5	349	1,056	1,042	2,100
45	169	7,359	506	7,867

Totals include two final verification calls. Increasing the available prefix reduces contractions on this example but increases operand regeneration. These are costs of particular selected runs, not minimum costs.

3 What the prefix estimate can certify without reuse

For a target contraction factor 2^p, the Euclidean-prefix lemma of the companion paper [2], combined with its three-call contraction (section "Three-call contractions"), uses $h = 4^p + 1$ ordered pairs. Building that prefix from its initial pair takes $h - 1 = 4^p$ Euclidean calls. Together they guarantee at least p bits of decrease. Suppose an analysis budgets a fresh prefix for each contraction, discards it afterwards, and accounts for progress only by these guaranteed factors. It may choose a different positive integer p_i at every stage.

Proposition 2 (Restricted accounting bound; checked in Lean)**.** *For any such finite list $p_1, \ldots, p_J$,*

$$\sum_{i=1}^{J} 4^{p_i} \;\geq\; 4 \sum_{i=1}^{J} p_i.$$

Consequently, if this accounting justifies a target decrease of n bits solely by $n \leq \sum_i p_i$, its charged prefix budget is at least $4n$.

Proof. For integers $p \geq 1$, $4p \leq 4^p$, by induction. Sum these inequalities and use the assumed target inequality. □

The Lean declarations, in `research/PrefixPolicies.lean`, are `ShortGCD.PrefixPolicies.rebuild_charge_ge_four_progress` and `ShortGCD.PrefixPolicies.target_charge`. This is a lower bound on the budget of this proof policy, not on actual executions: an actual successful run need not satisfy $n \leq \sum_i p_i$, since its contractions may be much stronger than their guaranteed factors, and the prototype reconstructs only its selected operands rather than a fresh prefix at every stage.

Two related attempts have the same limitation at the level of this estimate. Recomputing all prefix residues at every modulus replaces three calls per contraction by a budget growing with the prefix. Consuming disjoint prefix blocks while unwinding avoids repeated traversal, but spends a new block for each application of the residue-count lemma. For blocks with $h_i = 4^{p_i} + 1$ ordered pairs, a consecutive Euclidean prefix containing all these blocks takes $\sum_i h_i - 1 \geq \sum_i 4^{p_i}$ calls when $J \geq 1$. Under either accounting, certifying a decrease of cn bits, for fixed $c > 0$, costs at least linearly many calls in n. This leaves open reuse across overlapping blocks, stronger residue information, and schedules that exploit larger contractions.

4 A structural direction: bounded interfaces between calls

The following observation concerns computation occurrences, not numerical values identified whenever they happen to agree. It suggests a way to state the remaining issue more precisely; we do not claim it is new.

For a terminating evaluation, form its data-dependence graph. Give every primitive result and every local expression result an occurrence vertex. Give each call fresh formal-input ports and a return port, linked by copying edges to its actual arguments and its returned expression. Repeated uses of one formal input share that port. Keep separate vertices for unrelated occurrences even if their numerical values are equal. Consider the underlying undirected graph, optionally adding edges that make each primitive's operands and result a clique.

Proposition 3 (Call-tree decomposition; proof in this note)**.** *For each fixed finite first-order recursive program P without shared mutable state, these occurrence graphs have treewidth bounded by a constant depending only on P, independently of the input and recursion depth.*

Proof. Use the tree of dynamic function invocations, adding the head evaluation as its root. For each invocation, put in its bag its formal-input ports, the results of all subterm occurrences executed in its own body, its return port, and the formal-input and return ports of its immediate child invocations. The number of such occurrences and child calls is bounded by the finite body syntax, and each child has fixed arity. Taking the maximum over the finitely many bodies and the head gives a bound $C(P)$ on bag size.

Every primitive edge lies in the bag of the invocation executing it. Edges for argument and return copying lie in the caller or callee bag, and each interface port lies in both adjacent bags. Other local vertices lie only in their own bag. Thus the bags containing any vertex form a connected subtree, and every edge is covered. This is a tree decomposition of width at most $C(P) - 1$. □

Copy vertices may be removed by contracting copying edges, and unused tests and vertices may be deleted. These graph-minor operations preserve the existence of a bounded-width decomposition. Thus a small-cost execution gives a remainder derivation with a bounded-width occurrence graph as well. This argument does not allow the unrestricted identification of equal numerical values: that quotient need not preserve treewidth. Existing certificate files store numerical values, so measuring the width of their value-identified graphs would not by itself test this observation.

This leads to two concrete research questions.

1. Can the short prefix construction be organized into bounded-width occurrence graphs, with a decomposition and execution schedule realizable by one fixed program? The residue lemma supplies suitable operand pairs for each modulus but gives no compatibility between their choices at different moduli. Bounded treewidth alone is not asserted sufficient for such a compilation.

2. For every fixed width t, is there a $c_t > 0$ and an infinite family of inputs on which every remainder derivation of the gcd with occurrence treewidth at most t needs at least $c_t \log a$ calls? Using the decomposition above, such a theorem would imply the Main Conjecture for nondeterministic recursive programs [1], and hence for deterministic ones, which are a special case; the companion paper [2] leaves both open. No such lower bound is proved here. It would use a restriction on program computations beyond the homomorphism method of van den Dries and Moschovakis (see [2]).

Treewidth restrictions already play a role in circuit complexity; see, for example, Jansen and Sarma [4]. Their balancing results do not give a remainder-program speedup or the proposed lower bound. The immediate next test is to instrument the existing interpreter with occurrence identities, construct its invocation bags, and independently check the decomposition. That would make the proposed restriction concrete without assuming that recursion depth bounds storage or that equal values are interchangeable across inaccessible frames.

5 Reproduction and status

From the repository root:

```
lake build
lake env lean research/RegisterProgram.lean
lake env lean research/PrefixPolicies.lean
python3 research/rebuilding_program.py
python3 -O research/rebuilding_program.py
```

The machine correspondence is part of the normal library build, and its axiom dependencies are recorded in `validation/axioms.txt` and `research/register-axioms.txt`. The restricted budget lemma has its own audit in `research/prefix-policy-axioms.txt`. Prototype results are recorded in `rebuilding-validation.json`. The prototype and the call-tree decomposition are not formalized in Lean. The bounds established in the companion paper [2] are unchanged; one fixed sublogarithmic program remains open.

References

[1] Yiannis N. Moschovakis. *Abstract Recursion and Intrinsic Complexity*. Lecture Notes in Logic 48, Cambridge University Press / ASL, 2019. doi:10.1017/9781108234238.

[2] Jason Davies. *The Euclidean algorithm is not optimal among uniform processes*. Companion repository manuscript, October 2026.

[3] *Fixed-arity nondeterministic remainder programs: expanded proof and executable checks*. Research note accompanying the uniform-process construction, `research/nondeterministic-prefix.pdf`, October 2026.

[4] Maurice Jansen and Jayalal Sarma M.N. *Balancing Bounded Treewidth Circuits*, 2009. https://arxiv.org/abs/0910.1427.